

Скриптови езици и функционално програмиране - конспект

1. Въведение в Python. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.
2. Вход и изход. Аритметични операции и побитови операции с числа. Условни изрази. Условни конструкции. Цикли.
3. Функции. Ламбда изрази. Преобразуване на типове. Обхват на променливите.
4. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект.
5. Обработка на грешки и изключения.
6. Списъци, кортежи и речници.
7. Модули. Дефиниране и свързване на модули.
8. Струни. Работа със струни. Основни низови методи. Форматиране.
9. Съвпадение на образци. Основна конструкция. Кортежи в съвпадение на шаблони. Масиви в съпоставяне на шаблони. Речници в съпоставяне на образци. Класове по съпоставяне на шаблони. Предпазители или ограничения на модела. Задаване на псевдоними.
10. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове. Двоични файлове. Рафтов модул.
11. Работа с дати и часове. Модул за дата и час. Операции с дати.
12. Създаване на GUI. Използване на Tkinter.
13. Работа с функции. Първокласни функции. Анонимни функции. Рекурсия. Картографиране, филтриране, намаляване и изброяване на разбираня. Операторски модул.
14. Функционално програмиране. Идентифициране на парадигма. Процедурна и функционална парадигма. Използване на функционален хибрид. Създаване на обект. Класически пример за функционално програмиране.

Скриптови езици и функционално програмиране - конспект

15. Функции, итератори и генератори. Писане на чисти функции. Използване на низове. Използване на кортежи и именувани кортежи. Използване на генераторни изрази. Изследване на ограниченията на генераторите. Комбиниране на генераторни изрази. Почистване на необработени данни с функции на генератора.

16. Работа с колекции. Преглед на разновидностите на функциите. Работа с iterable. Синтактичен анализ на XML файл. Синтактичен анализ на файл на по-високо ниво. Сдвояване на елементи от последователност. Използване на функцията `iter()` изрично. Използване на `any()` и `all()`. Използване на `len()` и `sum()`. Използване на `reversed()` за промяна на реда. Използване на `enumerate()`.

17. Функции от по-висок ред. Използване на `max()` и `min()` за намиране на екстремум. Използване на ламбда форми на Python. Ламбда и ламбда изчисление. Използване на функцията `map()` за прилагане на функция към колекция. Работа с ламбда формуляри и `map()`. Използване на функцията `filter()` за предаване или отхвърляне на данни. Използване на `sorted()` за подреждане на данни. Писане на функции от по-висок ред.

18. Рекурсии и редукции. Прости числени рекурсии. Внедряване на оптимизация за опашно повикване.

19. Допълнителни техники за кортежи. Използване на непроменяем именуван набор като запис. Изграждане на именувани кортежи с функционални конструктори. Изчисляване на корелацията на ранг-порядък на Спийрман. Съвпадение на полиморфизъм и питоничен модел.

20. Модул `Itertools`. Работа с безкрайните итератори. Броене с `count()`. Повтаряне на цикъл със `cycle()`. Повтаряне на една стойност с `repeat()`. Използване на крайни итератори. Присвояване на числа с `enumerate()`. Текущи суми с натрупване. Комбиниране на итератори с функцията `chain()`.

21. Още техники на `Itertools`. Изброяване на декартово произведение. Намаляване на продукт.

22. Модул `Functools`. Функционални инструменти. Запомняне на предишни резултати с `lru_cache`. Дефиниране на класове с обща поръчка. Дефиниране на числови класове. Прилагане на частични аргументи с `partial()`. Намаляване на набори от данни с `reduction()`. Комбиниране на `map()` и `reduce()`. Използване на `reduce()` и `partial()`. Използване на `map()` и `reduce()` за дезинфекция на необработените данни. Използване на `groupby()` и `reduce()`.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

1. Програмиране в интерактивен режим

При извикването на интерпретатора без предаване на скриптов файл, като параметър в Python, се извежда покана:

```
$ python
Python 2.4.3 (#1, Nov 11 2010, 13:34:43)
[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Пример 1: *Да се въведе текст в подканата на Python и да се натисне Enter.*

```
>>> print "Hello, Python!"
```

Ако се използва новата версия на Python, тогава ще трябва да се използва оператор за печат със скоби, като се запише:

```
print ("Hello, Python!").
```

Във версия на Python 2.4.3 на екрана ще се изведе резултатът:

Hello, Python!

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

2. Програмиране в скриптов режим

При извикването на интерпретатора с параметър на скрипт в Python, характерното е че, докато се пише скрипта, интерпретаторът е активен, а след това се деактивира.

Пример 2: *Да се напише програма на Python в скрипт.*

Първи вариант

1. Във файл test.py се добавя следният код: **print "Hello, Python!,,**
2. Предполага се, че има интерпретатор на Python, зададен в променлива PATH.
3. Програмата се стартира: **\$ python test.py**
4. Резултатът е: **Hello, Python!**

Втори вариант

1. Модифицира се файлът от вариант 1- test.py:
`#!/usr/bin/python`
`print "Hello, Python!,,`
2. Предполага се, че има интерпретатор на Python, зададен в директорията **/usr/bin**.
3. Програмата се стартира:
\$ chmod +x test.py # This is to make file executable
\$/test.py
4. Резултатът е: **Hello, Python!**

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

3. Идентификатори на Python

Идентификаторът на Python е име, използвано за идентифициране на променлива, функция, клас, модул или друг обект. Идентификаторът започва с буква от А до Z или от а до z или долна черта (_), последвана от нула или повече букви, долни черти и цифри (0 до 9).

Python не позволява препинателни знаци като @, \$ и % в идентификаторите. Python е език за програмиране, чувствителен към малки и големи букви.

Пример 3: Manpower и manpower са два различни идентификатора в Python.

Конвенции за именуване за идентификатори на Python:

- Имената на класовете започват с **главна буква**. Всички останали идентификатори започват с малка буква.
- Ако идентификаторът започава с една водеща долна черта показва, че идентификаторът е **частен**.
- Ако идентификаторът започва с две водещи долни черти показва **силно частен** идентификатор.
- Ако идентификаторът също завършва с две долни черти в края, идентификаторът е **специално име**, дефинирано от език.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

4. Запазени думи

Следващият списък показва ключовите думи на Python. Това са запазени думи и не трябва да се използват като константа, променлива или други имена на идентификатори. **Всички ключови думи на Python съдържат само малки букви.**

Таблица 1. Запазени думи в Python

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield

5. Линии и отстъп

Python не предоставя скоби за обозначаване на блокове от код за дефиниции на класове и функции или контрол на потока. **Блоковете код се обозначават с отстъп на ред, който е строго наложен. Броят на интервалите в отстъпа е променлив, но всички изрази в блока трябва да имат еднакъв отстъп.**

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Пример 3:

```
if True:  
    print "True" else:  
    print "False,,
```

Следният блок обаче генерира грешка:

```
if True:  
    print "Answer" print "True"  
else:  
    print "Answer"  
    print "False“
```

В Python всички непрекъснати редове с отстъп със същия брой интервали образуват блок.

6. Python - видове променливи

Променливите не са нищо друго освен запазени места в паметта за съхранение на стойности. Това означава, че когато създавате променлива, запазвате малко място в паметта.

Въз основа на типа данни на променлива, интерпретаторът разпределя памет и решава какво може да се съхранява в запазената памет. Следователно, като присвоите различни типове данни на променливи, можете да съхранявате цели числа, десетични знаци или знаци в тези променливи.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

6.1. Присвояване на стойности на променливи

Променливите на Python не се нуждаят от изрична декларация, за да резервират място в паметта. Декларацията се случва автоматично, когато присвоите стойност на променлива. Знакът за равенство (=) се използва за присвояване на стойности на променливи. Операндът вляво от оператора равно, е името на променливата, а операндът вдясно от оператора равно, е стойността, съхранена в променливата.

Пример 4:

```
#!/usr/bin/python
```

```
counter = 100 # An integer assignment
```

```
miles = 1000.0 # A floating point
```

```
name = "John" # A string
```

```
print (counter)
```

```
print (miles)
```

```
print (name)
```

Резултат:

100

1000.0

John

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

6.2. Множествено присвояване

Python позволява да се присвои една стойност на няколко променливи едновременно.

Пример 5:

```
a=b=c=1
print (a) //извеждане на стойността на a
print (a, b, c) //извеждане на стойността на a, b, c
```

Резултат:

```
1          // принтиране на a
1 1 1      // принтиране на едновременно на трите стойности
```

Тук се създава целочислен обект със стойност 1, така, че и трите променливи се присвояват на едно и също място в паметта. Може също да се присвоят множество обекти на множество променливи.

Пример 6:

```
a,b,c = 1,2,"john"
print (a, b, c)
```

Резултат:

```
1 2 john
```

Тук два целочислени обекта със стойности 1 и 2 се присвояват съответно на променливите a и b, а един низ от обект със стойността "john" се присвоява на променливата c.

дисциплина: Скриптови езици и функционално програмиране

лектор: гл. ас. д-р инж. Димитричка Николаева

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

7. Стандартни типове от данни

Данните, съхранявани в паметта, могат да бъдат различни.

Пример: Възрастта на дадено лице се съхранява като числова стойност, а неговият адрес се съхранява като буквено-цифрови знаци.

В Python има различни стандартни типове данни, които се използват за дефиниране на възможните операции върху тях и метода на съхранение за всеки от тях. В Python има пет стандартни типа данни:

- Числа;
- Струна;
- Списък;
- Кортеж;
- Речник.

7.1. Присвояване

Създаване и променяне на променливи става предимно с оператора за присвояване. Този израз предоставя израз и име на променлива, които ще бъдат използвани за етикетирание на стойността на израза.

variable = expression

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Пример 7: Изрази за присвояване

example.py

```
#!/usr/bin/env python
# Computer the value of a block of stock
shares= 150
price= 3 + 5.0/8.0
value= shares * price
print (value)
```

Резултат: 543.75

7.2. Едновременно присвояване (задаване)

Има алтернативна форма на оператора за присвояване, която ни позволява да изчислим няколко стойности едновременно. Например:

<var>, <var>, ..., <var> = <expr>, <expr>, ..., <expr>

Този начин на присвояване е известен, като **едновременно възлагане**.

Семантично това казва на Python да оцени всички изрази от дясната страна и след това да присвои тези стойности на съответните променливи, намиращи се от лявата страна.

Подобен начин на присвояване изглежда странно, но може да се окаже забележително полезен.

Пример 8: Да се дефинират две променливи x и y. Да се намери сумата и разликата от двете променливи.

Пример 9: Да се дефинират две променливи x и y. Да се разменят стойностите им - т.е. стойността, която в момента се съхранява в x, да бъде в y, а стойността, която в момента е в y, да се съхранява в x.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Да предположим, че x и y имат стойности съответно 2 и 4.

Как ще се променят променливите?

Коментарните редове описват, какво се случва с променливите, докато тези два оператора се изпълняват:

#1) variables x y

#2) initial values 2 4 $x=y$

#3) final 4 4

Вижда се как първият оператор заличава присвояването на x на y във втората стъпка. Нека да добавим трета променлива `temp` и да преминем през тази последователност, за да видим как работи.

#1) variables x y `temp`

#2) initial values 2 4 no value yet `temp = x`

→ т.е начални стойности 2 4 все още няма стойност `temp = x`

Както можете да видите от крайните стойности на x и y , размяната е успешна в този случай.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Този вид трипосочно разбъркване е често срещано в други езици за програмиране. В Python операторът за едновременно присвояване предлага елегантна алтернатива. Ето по-опростен еквивалент на Python:

Първоначалната стойност на x чрез присвояване на стойността на y . Като, завършва след това с две копия на оригиналната стойност на y .

За въвеждане на допълнителна променлива, която временно запомня x , $y = y, x$

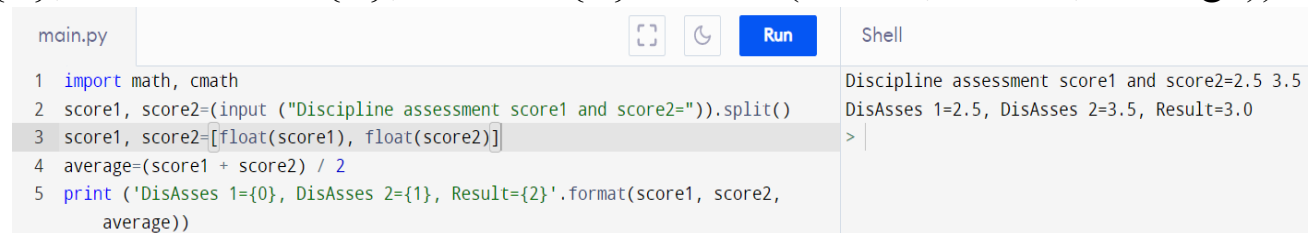
Тъй като присвояването е едновременно, то избягва изтриването на една от първоначалните стойности.

Едновременното присвояване може също да се използва за получаване на множество стойности от потребителя в един вход.

Пример 9: Да се създаде програма за осредняване на резултатите от 2 изпита:

Вариант 1

```
import math, cmath
score1, score2=(input ("Discipline assessment score1 and score2=")).split()
score1, score2=[float(score1), float(score2)]
average=(score1 + score2) / 2
print ('DisAsses 1={0}, DisAsses 2={1}, Result={2}'.format(score1, score2, average))
```



```
main.py
1 import math, cmath
2 score1, score2=(input ("Discipline assessment score1 and score2=")).split()
3 score1, score2=[float(score1), float(score2)]
4 average=(score1 + score2) / 2
5 print ('DisAsses 1={0}, DisAsses 2={1}, Result={2}'.format(score1, score2,
    average))

Shell
Discipline assessment score1 and score2=2.5 3.5
DisAsses 1=2.5, DisAsses 2=3.5, Result=3.0
>
```

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Програмата подканва за два резултата, разделени с празна позиция. Получихме стойност за всяка от променливите с един замах. Примерът използва само две стойности, но може да се обобщи за произволен брой входове. Разбира се, може просто да получим входа от потребителя, използвайки отделни оператори за въвеждане:

Вариант 2

```
# avg.py
score1 = float(input("Discipline assessment 1: "))
score2 = float(input("Discipline assessment 2: "))
average=(score1 + score2) / 2
print ("The average of the scores is:", average)
```

Вход: Discipline assessment 1: 2.5, Discipline assessment 2: 3.5

Изход: The average of the scores is: 3

main.py	  	Shell
<pre>1 score1 = float(input("Discipline assessment 1: ")) 2 score2 = float(input("Discipline assessment 2: ")) 3 average=(score1 + score2) / 2 4 print ("The average of the scores is:", average)</pre>		<pre>Discipline assessment 1: 2.5 Discipline assessment 2: 3.5 The average of the scores is: 3.0 ></pre>

В някои отношения това може да е по-доре, тъй като отделните подкани са по-информативни за потребителя. В този пример решението кой подход да се предприеме до голяма степен е въпрос на вкус. Понякога получаването на множество стойности в един вход осигурява по-интуитивен потребителски интерфейс.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

8. Константи

Константата е вид променлива, която съдържа стойности, които не могат да бъдат променяни. В действителност рядко се използват константи в Python. Обикновено се декларираат и присвояват на различен модул/файл.

Пример 10:

#Деклариране на константи във файл наречен - **constant.py**

PI = 3.14

GRAVITY = 9.8

Импортиране в основния файл на тези константи

#inside main.py we import the constants import constant

print(constant.PI)

print(constant.GRAVITY)

9. Видове оператори

Езикът Python поддържа следните типове оператори:

- *Аритметични оператори;*
- *Оператори за сравнение (релационни);*
- *Оператори за присвояване;*
- *Логически оператори;*
- *Побитови оператори;*
- *Членски оператори (Membership);*
- *Оператори на идентичност (Identity).*

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.1. Аритметични оператори на Python

Да приемем, че променлива $a = 10$ и променлива $b = 20$, тогава:

Таблица:

оператор	описание	пример
+ Addition	Добавя стойности от двете страни на оператора.	$a + b = 30$
- Subtraction	Изважда десния операнд от левия операнд.	$a - b = -10$
* Multiplication	Умножава стойностите от двете страни на оператора.	$a * b = 200$
/ Division	Разделя левия операнд на десен операнд.	$b / a = 2$
% Modulus	Разделя левия операнд на десен операнд и връща остатъка.	$b \% a = 0$
** Exponent	Извършва експоненциално изчисление на операторите	$a ** b =$ 10 на степен 20
// Floor Division	Деление на операндите, при което резултатът е частното, в което се премахват цифрите след десетичната запетая. Но ако един от операндите е отрицателен, резултатът е занижен, т.е. закръгля се от нула (към отрицателна безкрайност)	$9 // 2 = 4,$ $9.0 // 2.0 = 4.0,$ $-11 // 3 = -4,$ $-11.0 // 3 = -4.0$

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.2. Оператори за сравнение на Python

Тези оператори сравняват стойностите от двете им страни и решават връзката между тях. Те се наричат още релационни оператори.

Таблица:

оператор	описание	пример
==	Ако стойностите на два операнда са равни, тогава условието става вярно.	(a == b) не е true
!=	Ако стойностите на два операнда не са равни, тогава условието става вярно.	(a != b) е true
<>	Ако стойностите на два операнда не са равни, тогава условието става вярно.	(a <> b) е true Това е подобно на оператора !=
>	Ако стойността на левия операнд е по-голяма от стойността на десния операнд, тогава условието става вярно.	(a > b) не е true
<	Ако стойността на левия операнд е по-малка от стойността на десния операнд, тогава условието става вярно.	(a < b) е true
>=	Ако стойността на левия операнд е по-голяма или равна на стойността на десния операнд, тогава условието става вярно.	(a >= b) не е true
<=	Ако стойността на левия операнд е по-малка или равна на стойността на десния операнд, тогава условието става вярно.	(a <= b) е true

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.3. Оператори за присвояване на Python

Таблица:

оператор	описание	пример
=	Присвоява стойности от десните операнди на левия операнд	$c = a + b$ присвоява стойност на $a + b$ на c
+= Add AND	Той добавя десния операнд към левия операнд и присвоява резултата на левия операнд	$c += a$ е еквивалентно на $c = c + a$
-= Subtract AND	Той изважда десния операнд от левия операнд и присвоява резултата на левия операнд	$c -= a$ е еквивалентно на $c = c - a$
*= Multiply AND	Той умножава десния операнд с левия операнд и присвоява резултата на левия операнд	$c *= a$ е еквивалентно на $c = c * a$
/= Divide AND	Той разделя левия операнд с десния операнд и присвоява резултата на левия операнд	$c /= a$ е еквивалентно на $c = c / a$
%= Modulus AND	Той взема модул, използвайки два операнда и присвоява резултата на левия операнд	$c \% = a$ е еквивалентно на $c = c \% a$
**= Exponent AND	Извършва експоненциално (мощност) изчисление на оператори и присвоява стойност на левия операнд	$c ** = a$ е еквивалентно на $c = c ** a$
//= Floor Division	Той извършва разделяне на етажа на операторите и присвоява стойност на левия операнд	$c //= a$ е еквивалентно на $c = c // a$

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.4. Побитови оператори на Python

Побитовият оператор работи върху битове и изпълнява бит по битова операция.

Да приемем, че $a = 60$; и $b = 13$;

В двоичен формат техните стойности са съответно 0011 1100 и 0000 1101.

Побитовите оператори, поддържани от езика Python при посочените стойности:

$a = 0011\ 1100$

$b = 0000\ 1101$

$a \& b = 0000\ 1100$

$a | b = 0011\ 1101$

$a \wedge b = 0011\ 0001$

$\sim a = 1100\ 0011$

Следващата таблица изброява побитовите оператори, поддържани от езика Python с пример за всеки от тях, ние използваме горните две променливи (a и b) като операнди:

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Таблица:

оператор	описание	пример
& Binary AND	Операторът копира a bit към резултата, ако съществува и в двата операнда	(a & b) (означава 0000 1100)
Binary OR	Копира a bit, ако съществува в който и да е операнд.	(a b) = 61 (означава 0011 1101)
^ Binary XOR	Той копира бита, ако е зададен в един операнд, но не и в двата.	(a ^ b) = 49 (означава 0011 0001)
~ Binary Ones Complement	Той е унарен и има ефект на „обръщане“ на битове.	(~a) = -61 (означава 1100 0011 във формата за допълване на 2 поради двоично число със знак.)
<< Binary Left Shift	Стойността на левия операнд се премества наляво с броя битове, посочени от десния операнд.	a << 2 = 240 (означава 1111 0000)
>> Binary Right Shift	Стойността на левия операнд се премества надясно с броя битове, посочени от десния операнд.	a >> 2 = 15 (означава 0000 1111)

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.5. Логически оператори на Python

Таблица:

оператор	описание	пример
and Logical	Ако и двата операнда са верни, тогава условието става	(a and b)
AND	True	e true
or Logical OR	Ако някой от двата операнда е различен от нула, тогава условието става вярно.	(a or b) e true
not Logical NOT	Използва се за обръщане на логическото състояние на неговия операнд.	Not (a and b) e false

Таблица:

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.5.1. Логически оператори и Методът `bool()`:

Логическите оператори са: `and`, `or`, `not`.

BOOL() В PYTHON

се използва за връщане или преобразуване на стойност в булева стойност, т.е. `True` или `False`, като се използва стандартната процедура за тестване на истинността.

Синтаксис: `bool([x])`

Методът `bool()`, като цяло приема само един параметър, върху който може да се приложи стандартната процедура за проверка на истинността. Ако не бъде предаден параметър, по подразбиране той връща `False`.

Така че предаването на параметър не е задължително. Може да върне една от двете стойности.

- Връща `True`, ако предаденият параметър или стойност е `True`.
- Връща `False`, ако предаденият параметър или стойност е `False`.

Ето няколко случая, в които методът `bool()` на Python връща `false`. С изключение на тези всички други стойности връщат `True`.

- Ако е предадена стойност `False`.
- Ако е предаден `None`.
- Ако е предадена празна последователност, като `()`, `[]`, `“` и т.н.
- Ако нула се предава в произволен числов тип, като `0`, `0.0` и т.н.
- Ако е предадено празно съпоставяне, като `{}`.
- Ако обекти от класове имат метод `bool()` или `len()`, връща `0` или `False`. Методът `bool()`

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.5.2. Конверсия на типове в Python

Python дефинира функции за преобразуване на типове за директно преобразуване на един тип данни в друг, което е полезно в ежедневно и конкурентно програмиране. Тази статия е насочена към предоставяне на информация за определени функции за преобразуване.

1. **int(a,base)** : Тази функция преобразува всеки тип данни в цяло число. „base“ указва базата, в която е низът, ако типът данни е низ.
2. **float()** : Тази функция се използва за преобразуване на всеки тип данни в число с плаваща запетая.
3. **ord()** : Тази функция се използва за преобразуване на символ в цяло число.
4. **hex()** : Тази функция преобразува цяло число в шестнадесетичен низ.
5. **oct()** : Тази функция преобразува цяло число в осмичен низ.

9.5.3. Loop statement

Езикът за програмиране Python предоставя следните типове цикли:

Таблица:

No	Loop Type & Description
1	while loop - Повтаря израз или група от изрази, докато дадено условие е TRUE. Той тества условието преди да изпълни тялото на цикъла.
2	for loop - Изпълнява последователност от изрази няколко пъти и съкращава кода, който управлява променливата на цикъла.
3	nested loops (вложен) - Можете да използвате един или повече цикли във всеки друг цикъл while, for или do..while.

9.5.3.1. Loop Control Statements

Контролните оператори на цикъла променят изпълнението от нормалната му последователност. Когато изпълнението напусне обхват, всички автоматични обекти, които са били създадени в този обхват, се унищожават. Python поддържа следните контролни оператори. Щракнете върху следните връзки, за да проверите техните подробности. Нека преминем накратко през операторите за контрол на цикъла

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

9.6. Побитови оператори:

Нека $x = 10$ (0000 1010 в двоично) и $y = 4$ (0000 0100 в двоично)

Предимство на операторите на Python

Комбинацията от стойности, променливи, оператори и извиквания на функции се нарича израз. Интерпретаторът на Python може да оцени валиден израз.

Пример:

```
>>> 5 - 7
```

```
-2
```

Тук $5-7$ е израз. В израз може да има повече от един оператор.

За оценка на този тип изрази има правило за приоритет в Python. Той ръководи реда, при които се извършват операции.

Приоритет на оператора:

Когато израз съдържа повече от един оператор, редът на оценка зависи от реда на операциите.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Пример: Изчисляване на индекс на телесна маса в Python

```
#Define the constants
METER = 100
#Read the inputs from user
height = float(input("Enter your height in Centimeters: "))
weight = float(input("Enter your weight in Kg: "))
temp = height / METER
#Calculate the BMI
bmi = weight / (temp*temp)
#Display the result
print("Your Body Mass Index is: ", "%d"%(bmi))
```

Резултат:

```
C:\Python\programs>python program.py
Enter your height in Centimeters: 105
Enter your weight in Kg: 69
Your Body Mass Index is: 62
```

10. Клаузите else и elif

Сега знаете как да използвате оператор if за условно изпълнение на един оператор или блок от няколко оператора. Време е да разберете какво друго можете да направите. Понякога искате да оцените условие и да вземете един път, ако е вярно, но да посочите алтернативен път, ако не е. Това се постига с друга клауза:

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

if <expr>:

<statement(s)> else:

<statement(s)>

if <expr> е true, първият пакет се изпълнява, а вторият се пропуска.

Ако <expr> е false, първият пакет се пропуска и вторият се изпълнява. Така или иначе, изпълнението се възобновява след втория пакет. И двата пакета са дефинирани чрез отстъп, както е описано по-горе.

10.1. Едноредни изявления if

Обичайно е да се пише if <expr> на един ред и <изявление> с отстъп на следния ред по следния начин:

if <expr>:

<statement>

Но е допустимо да се напише цял оператор if на един ред. Следното е функционално еквивалентно на примера по-горе:

if <expr>: <statement>

Може дори да има повече от един <statement> на един и същи ред, разделени с точка и запетая:

if <expr>: <statement_1>; <statement_2>; ...; <statement_n>

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Но какво означава това? Има две възможни интерпретации:

Ако `<expr>` е true, изпълнете `<statement _1>`.

След това безусловно изпълняване на `<statement _2> ... <statement _n>`, независимо дали `<expr>` е true или not.

Ако `<expr>` е вярно, изпълняване на всички `<statement _1> ... <statement _n>`.

В противен случай не се изпълнява нито един от тях.

Python приема последната интерпретация.

Точката и запетаята, разделяща `<statements>`, има по-висок приоритет от двоеточието след `<expr>`

Това на компютърен жаргон казва, че точката и запетая се свързват по-здраво от двоеточието. По този начин `<statements>` се третираат като пакет и или всички `statements`-и се изпълняват, или никой от тях не се изпълнява.

10.2. Break & continue statements

Python предоставя оператори `break` и `continue` за справяне с такива ситуации и за добър контрол върху своя цикъл. Ще обсъди изявленията за прекъсване, продължаване и преминаване, налични в Python.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

10.3. break Statement:

Операторът `break` в Python прекратява текущия цикъл и възобновява изпълнението при следващия оператор, точно както традиционния `break` в програмният език C.

Най-честата употреба за прекъсване е, когато се задейства някакво външно условие, изискващо прибързано излизане от цикъл. Операторът `break` може да се използва както в цикли `while`, така и във `for`.

```
#!/usr/bin/python
for letter in 'Python':
    if letter == 'h':
        break
print ("Current Letter :", letter)
```

Резултат:

Current Letter : h

main.py	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 for letter in 'Python': 4 # First Example 5 if letter == 'h': 6 break 7 else: 8 print ("Current Letter :", letter) 9 var = 10 10 # Second Example 11 while var > 0: 12 var = var -1 13 if var == 5: 14 break 15 else: 16 print ("Current Letter :", var) 17 print ("Good bye!")</pre>	<pre>Current Letter : P Current Letter : y Current Letter : t Current Letter : 9 Current Letter : 8 Current Letter : 7 Current Letter : 6 Good bye! ></pre>

```
#!/usr/bin/python
for letter in 'Python':
    # First Example
    if letter == 'h':
        break
    else:
        print ("Current Letter :", letter)
var = 10
# Second Example
while var > 0:
    var = var -1
    if var == 5:
        break
    else:
        print ("Current Letter :", var)
print ("Good bye!")
```

Резултат:

Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : 9
Current Letter : 8
Current Letter : 7
Current Letter : 6
Good bye!

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

10.4. continue Statement:

Операторът continue в Python връща контролата в началото, в сравнение със цикъла while. Инструкцията за продължаване отхвърля всички останали изрази в текущата итерация на цикъла и премества контролата обратно в началото на цикъла.

Инструкцията за продължаване може да се използва както в цикли while, така и в for.

```
#!/usr/bin/python
for letter in 'Python':
    # First Example
    if letter == 'h':
        continue
    else:
        print ("Current Letter :", letter)
var = 10
    # Second Example
while var > 0:
    var = var -1
    if var == 5:
        continue
    else:
        print ("Current Letter :", var)
print ("Good bye!")
```

Резултат:

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Current Letter : 9
Current Letter : 8
Current Letter : 7
Current Letter : 6
Current Letter : 4
Current Letter : 3
Current Letter : 2
Current Letter : 1
Current Letter : 0
Good bye!
```

main.py	Run	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 for letter in 'Python': 4 # First Example 5 if letter == 'h': 6 continue 7 else: 8 print ("Current Letter :", letter) 9 var = 10 10 # Second Example 11 while var > 0: 12 var = var -1 13 if var == 5: 14 continue 15 else: 16 print ("Current Letter :", var) 17 print ("Good bye!")</pre>		<pre>Current Letter : P Current Letter : y Current Letter : t Current Letter : o Current Letter : n Current Letter : 9 Current Letter : 8 Current Letter : 7 Current Letter : 6 Current Letter : 4 Current Letter : 3 Current Letter : 2 Current Letter : 1 Current Letter : 0 Good bye! ></pre>

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

10.5. else е израз, използван с цикли:

Python поддържа да има оператор else, свързан с оператори на цикъл.

- Ако операторът else се използва с цикъл for, операторът else се изпълнява, когато цикълът е изчерпал итерацията на списъка.
- Ако операторът else се използва с цикъл while, операторът else се изпълнява, когато условието стане невярно.

Пример:

Да се напише задача, която търси прости числа в интервала от 10 до 20, като се използва комбинацията от оператор else с оператор for:

```
#!/usr/bin/python
```

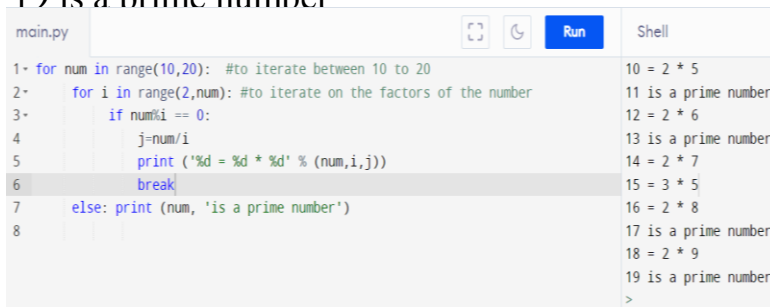
```
for num in range(10,20): #to iterate between 10 to 20
    for i in range(2,num): #to iterate on the factors of the number
        if num%i == 0:
            j=num/i
            print ('%d = %d * %d' % (num,i,j))
            break
    else: print (num, 'is a prime number')
```

Забележка: (първото %d) - указва стойността на първата променлива посочена в скобите т.е. num, следващото (второто %d) - стойността на i (третото %d) - стойността на j.

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Резултат:

```
10 = 2 * 5
11 is a prime number
12 = 2 * 6
13 is a prime number
14 = 2 * 7
15 = 3 * 5
16 = 2 * 8
17 is a prime number
18 = 2 * 9
19 is a prime number
```



```
main.py Run Shell
1- for num in range(10,20): #to iterate between 10 to 20
2-     for i in range(2,num): #to iterate on the factors of the number
3-         if num%i == 0:
4-             j=num/i
5-             print ('%d = %d * %d' % (num,i,j))
6-             break
7-         else: print (num, 'is a prime number')
8- 
```

```
10 = 2 * 5
11 is a prime number
12 = 2 * 6
13 is a prime number
14 = 2 * 7
15 = 3 * 5
16 = 2 * 8
17 is a prime number
18 = 2 * 9
19 is a prime number
>
```

По подобен начин можете да използвате оператор `else` с цикъл `while`.

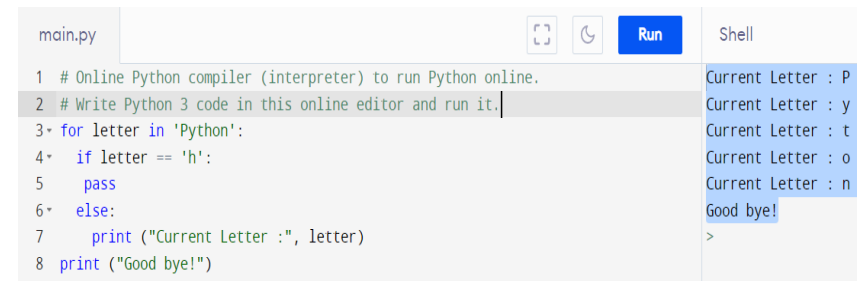
pass Statement:

Операторът `pass` в Python се използва, когато израз се изисква синтактично, но не искате да се изпълнява никаква команда или код.

Операцията `pass` е нулева операция; нищо не се случва, когато се изпълнява.

Пропускът е полезен и на места, където вашият код в крайна сметка ще отиде, но все още не е написан:

```
#!/usr/bin/python
for letter in 'Python':
    if letter == 'h':
        pass
    else:
        print ("Current Letter :", letter)
print ("Good bye!")
```



```
main.py Run Shell
1 # Online Python compiler (interpreter) to run Python online.
2 # Write Python 3 code in this online editor and run it.
3- for letter in 'Python':
4-     if letter == 'h':
5-         pass
6-     else:
7-         print ("Current Letter :", letter)
8- print ("Good bye!")
```

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Good bye!
>
```

Резултат ще е следния:

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Good bye!
```


Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

ПОКАЗВАНЕ НА ПРОСТИ ЧИСЛА

#Програма на Python с която се извеждат
всички прости числа в интервала от 900 до
1000

```
lower = 900
upper = 1000
print("Prime numbers between",
lower, "and", upper, "are:")
for num in range(lower, upper + 1):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)
```

Резултат:

Prime numbers between 900
and 1000 are: 907

911
919
929
937
941
947
953
967
971
977
983
991
997

main.py	Run	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 lower = 900 4 upper = 1000 5 print("Prime numbers between", lower, "and", upper, "are:") 6 for num in range(lower, upper + 1): 7 if num > 1: 8 for i in range(2, num): 9 if (num % i) == 0: break 10 else: print(num)</pre>		<pre>Prime numbers between 900 and 1000 are: 907 911 919 929 937 941 947 953 967 971 977 983 991 997</pre>

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Прости числа

Простото число е цяло число, по-голямо от 1, което се дели на 1 и на себе си. Първите няколко прости числа са:

2, 3, 5, 7, 11, 13, 17, 19, 23 и 29.

За да се провери дали едно число, например num е просто, първо се разделя на 2.

Ако резултата от $\text{num} \% 2 = 0$, то num не е просто.

Иначе е просто число.

Ще се демонстрират 2 варианта на задачата

Пример: (using for statement)

```
# To display all the prime numbers within a given interval [Lower, Upper]
lower = int(input("Enter Lower Limit: "))
upper = int(input("Enter Upper Limit: "))
print("Prime numbers between", lower, "and", upper, "are:")
for num in range(lower, upper + 1):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)
```

Резултат:

Enter Lower Limit: 20

Enter Upper Limit: 50

Prime numbers between 20 and 50 are:

23

29

31

37

41

43

47

>>>

main.py	Run	Shell
1 # Online Python compiler (interpreter) to run Python online.		Enter Lower Limit: 20
2 # Write Python 3 code in this online editor and run it.		Enter Upper Limit: 50
3 # To display all the prime numbers within a given interval [Lower, Upper]		Prime numbers between 20 and 50 are:
4 lower = int(input("Enter Lower Limit: "))		23
5 upper = int(input("Enter Upper Limit: "))		29
6 print("Prime numbers between", lower, "and", upper, "are:")		31
7 for num in range(lower, upper + 1):		37
8 if num > 1:		41
9 for i in range(2, num):		43
10 if (num % i) == 0: break		47
11 else: print(num)		>

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Пример: (using while statement)

```
# Python program to display all the prime numbers within a given interval [Lower, Upper] using while statement
lower = int(input("Enter Lower Limit: "))
upper = int(input("Enter Upper Limit: "))
print("Prime numbers between", lower, "and", upper, "are:")
num=lower
while(num<=upper):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)
        num=num+1
```

Резултат:

Enter Lower Limit: 50
Enter Upper Limit: 100
Prime numbers between 50 and 100 are:

53
59
61
67
71
73
79
83
89
97
>>>

main.py	Run	Shell
1 # Online Python compiler (interpreter) to run Python online.		Enter Lower Limit: 50
2 # Write Python 3 code in this online editor and run it.		Enter Upper Limit: 100
3 lower = int(input("Enter Lower Limit: "))		Prime numbers between 50 and 100 are:
4 upper = int(input("Enter Upper Limit: "))		53
5 print("Prime numbers between", lower, "and", upper, "are:")		59
6 num=lower		61
7 while(num<=upper):		67
8 if num > 1:		71
9 for i in range(2, num):		73
10 if (num % i) == 0: break		79
11 else: print(num)		83
12 num=num+1		89
13		97

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

РЕШАВАНЕ НА КВАДРАТИЧНО УРАВНЕНИЕ

В алгебрата квадратно уравнение е всяко уравнение от вида $ax^2 + bx + c = 0$, където x е неизвестно, а a , b и c са известни числа. Числата a , b и c са коефициентите на уравнението.

Примери: $x^2 + 5x + 6 = 0$, $4x^2 + 5x + 8 = 0$.

Стойностите на x , които удовлетворяват уравнението, се наричат решения или корени на уравнението. Квадратното уравнение винаги има два корена. Може да се намери чрез изчисляване на $d = b^2 - 4ac$. Стойността на d е нула или +ve или -ve.

Ако $d = 0$, тогава корените са реални и равни. Те са $\text{корен1} = -b/(2a)$ и $\text{корен2} = -b/(2a)$.

Ако $d > 0$, тогава корените са реални и различни. Те са $\text{корен1} = -b + \sqrt{d}/(2a)$ и $\text{корен2} = -b - \sqrt{d}/(2a)$.

Ако $d < 0$, тогава корените са комплексни или имагинерни. Корените са от вида $p + iq$ и $p - iq$, където p е реалната част от корена, а q е въображаемата част от корена. Реалната част на корена е $p = -b/(2a)$, а имагинерната част е $q = \sqrt{-d}/(2a)$.

main.py	  	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 import math,cmath 4 a,b,c= (input("Enter a, b and c: ")).split() 5 a,b,c =[int(a),int(b),int(c)] 6 d = (b**2) - (4*a*c) 7 if d==0: 8 sol1 = -b/(2*a) 9 sol2 = -b/(2*a) 10 print("Roots are real and equal") 11 elif d>0: 12 sol1 = (-b- math.sqrt(d))/(2*a) 13 sol2 = (-b+math.sqrt(d))/(2*a) 14 print("Roots are real and different") 15 elif d<0: 16 sol1 = (-b-cmath.sqrt(d))/(2*a) 17 sol2 = (-b+cmath.sqrt(d))/(2*a) 18 print("Roots are imaginary") 19 print('{0} and {1}'.format(sol1,sol2))</pre>		<pre>Enter a, b and c: 1 4 4 Roots are real and equal -2.0 and -2.0 > </pre>

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Пример:

```
# Finding the roots of quadratic equation  $ax^2 + bx + c = 0$  # import
math and complex math modules
import math,cmath
a,b,c=(input("Enter a, b and c: ")).split()
a,b,c=[int(a),int(b),int(c)]
d = (b**2) - (4*a*c)
if d==0:
    sol1 = -b/(2*a)
    sol2 = -b/(2*a)
    print("Roots are real and equal")
elif d>0:
    sol1 = (-b- math.sqrt(d))/(2*a)
    sol2 = (-b+math.sqrt(d))/(2*a)
    print("Roots are real and different")
elif d<0:
    sol1 = (-b-cmath.sqrt(d))/(2*a)
    sol2 = (-b+cmath.sqrt(d))/(2*a)
    print("Roots are imaginary")
print('{0} and {1}'.format(sol1,sol2))
```

Резултат:

Enter a, b and c: 1 4 4 Roots are real and equal
-2.0 and -2.0

>>>

Enter a, b and c: 1 5 6 Roots are real and different
-3.0 and -2.0

>>>

Enter a, b and c: 1 2 3 Roots are imaginary
(-1-1.4142135623730951j) and (-
1+1.4142135623730951j)

>>>

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009

Тема 2. Програмиране в интерактивен и скриптов режим. Идентификатори на Python. Конвенции за именуване на идентификатори на Python. Запазени думи. Променливи. Присвояване на стойности на променливи. Множествено присвояване. Стандартни типове от данни. Константи. Оператори – аритметични, побитови, логически, за сравнение, за присвояване. Конверсия на типове в Python.

Беше на 35 страница!!!

Едномерно произволно обикаляне Елементарен пример за произволно обикаляне е случайното обикаляне на целочислената права, която започва от 0 и на всяка стъпка се движи +1 или -1 с еднаква вероятност. Така че нека се опитаме да приложим 1-D произволната разходка в Python.

```
filter_none edit play_arrow brightness_4
# Python code for 1-D random walk.
import random
import numpy as np
import matplotlib.pyplot as plt
# Probability to move up or down
prob = [0.05, 0.95]
# statically defining the starting position
start = 2
positions = [start]
# creating the random points
rr = np.random.random(1000)
downp = rr < prob[0]
upp = rr > prob[1]

for idownp, iupp in zip(downp, upp):
    down = idownp and positions[-1] > 1
    up = iupp and positions[-1] < 4
    positions.append(positions[-1] - down + up)
# plotting down the graph of the random walk in 1D
plt.plot(positions)
plt.show()
```